Ders 7 Genişletilmiş Notu: Strategy, State, Template Method ve Visitor Kalıpları

Ders 7 Genişletilmiş Notu: Strategy, State, Template Method ve Visitor Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Strategy Kalıbı
- 2. Strategy Örneği: Uluslararası Kişisel Finans Programı
- 3. State Kalıbı
- 4. State Örneği: Kişisel Bilgi Yöneticisi
- 5. Template Method Kalıbı
- 6. Template Method Örneği: Veritabanından Bağımsız Sorgu
- 7. Visitor Kalıbı
- 8. Visitor Örneği: Proje Yönetim Yazılımı
- Örnek Sorular
- Dersin Toparlaması

1. Strategy Kalıbı

Geçen ders davranışsal kalıplardan Command, Iterator, Memento ve Observer işlenmiş, son olarak Observer üzerine konuşulmuştu. Bu derse davranışsal kalıplardan Strategy ile devam edildi.

1.1 Amaç

Strategy kalıbının amacı, aynı iş için farklı algoritmalar varken bu işin yapılmasını isteyen tarafı (istekçiyi) algoritmalardan soyutlamaktır. Bunun kazancı, kullanılan algoritmanın çalışma anında değiştirilebilmesidir. Hoca bunu şöyle tekrarladı: yapılacak bir iş var ve bu işin yapılabileceği farklı algoritmalar var; ama işin yapılmasını belirli bir algoritmaya sıkı bağlamak istemiyoruz, gevşek bağlamak (loosely coupled) istiyoruz ki çalışma anında bir algoritmadan diğerine geçebilelim. Bazı algoritmalar bazı durumlarda daha iyidir; o anki durum çözümlenip en uygun algoritma seçilebilir ve bu karar çalışma anında verilebilir. Bu, ancak bağlaşım iş ile algoritmalar birbirinden soyutlanarak kurulursa mümkündür.

1.2 Dörtlü Çete'nin örneği: satır bölmeleme

Bir kelime işlemci (metin editörü) yazdığımızı düşünelim. Sayfa sınırları bellidir ve satır sonuna gelindiğinde uzun bir kelimenin nasıl bölüneceğine karar vermek gerekir; buna satır bölmeleme (line breaking) denir. Bunun birden fazla yöntemi vardır ve biz hepsini desteklemek, kullanıcının seçeneklere girip "artık şöyle çalış" diyerek yöntemi değiştirebilmesini istiyoruz.

Hoca bunu ekranda gösterdi (ekrandan, 00:20:00). Word'de sayfa genişliği küçültülünce bir kelime doğrudan alt satıra atlıyordu; bu, boşluktan bölmeleme algoritmasıdır. WordPad'in "Görünüm" sekmesindeki "Sözcük kaydır" seçeneğinde üç seçenek vardır. Birincisi kaydırma yoktur: sayfa yapısına bakmadan Enter karakterine kadar her şey tek bir paragraf olarak tek

satıra yazılır. İkincisi pencereye göre kaydırmadır: pencerenin boyutu değiştirilince pencere nerede bitiyorsa oradaki kelimeden itibaren bölünür. Bu sırada program boşluk görünce yeni kelimeyi aşağı geçirir; ama hoca bir kelimenin içine elle bir heceleme (kesme) işareti koyunca bölmenin oradan yapıldığını gösterdi. Hoca çeşitli varyasyonlar olduğunu ve farklı rapor kurallarında bunların farklı biçimlerde istenebileceğini söyledi. Soru, bu bölmeleme biçimlerini yalnız ayarlardan seçerek değiştirebileceğimiz bir tasarımın nasıl yapılacağıdır.

Dörtlü Çete'nin önerdiği çözümde yapılacak işle ilgili soyut bir "satır bölmeleme" sınıfı hazırlanır (bu bir arayüz de olabilir) ve onda soyut bir "bölmele" metodu tanımlanır; yani yapılacak iş tanımlanır. İşin farklı yapılma biçimleri bu soyut sınıfın somut alt sınıflarında (arayüz seçilirse arayüzü gerçekleyen sınıflarda) kodlanır. Hoca, varsayılan bir davranış kodlanmak istenirse soyut sınıf seçilip bölmele metodunun somut yapılabileceğini, ama genelde arayüzün tercih edildiğini ve metotların tamamen gerçekleyen sınıflarda yazıldığını söyledi. Somut sınıfları, işi nasıl yaptıklarını anlatan uygun adlarla adlandırmak en doğrusudur.

Uygulama tarafında metin düzenleme sırasında bölmeleme gerekince bu iş "bölmeleyici" üye alanı üzerinden yaptırılır. Bu üye alana hangi alt sınıftan nesne verildiyse bölmeleme onun yaptığı biçimde yürür. Herhangi bir anda, örneğin kesmeli bölmeden vazgeçip kelimeden bölmeye geçmek istenirse, yapılacak tek şey bölmeleyici üyesine istenen türde bölmeleme yapan bir nesne vermektir.

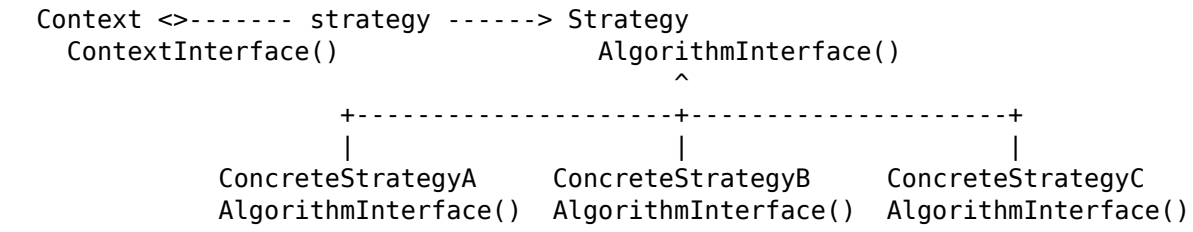
Hoca bir önceki slayttaki şemayı Dörtlü Çete'nin örneğinden Türkçeye çevirerek yazdığını, genelde bunu yapmadığını belirtti. Slaytlardaki Dörtlü Çete şemaları eski gösterimle çizilmiştir; hoca sınıf şemalarının güncel UML 2.0 standardına göre çizilmesi gerektiğini hatırlattı.

> **Ek bilgi (derste anlatılmadı):** Dörtlü Çete kitabı UML standartlaşmadan önce yazıldığı için şemalarında büyük olasılıkla OMT gösterimi kullanılır. Bu gösterimde örneğin ilişkinin çokluğu ok ucundaki dolu ya da boş yuvarlakla gösterilir; UML'de ise çokluk ilişki çizgisinin ucuna sayı ya da * olarak yazılır ve kalıtım içi boş üçgen okla gösterilir.

1.3 Kalıp yapısı ve bileşenler

Kalıbın genel yapısında işlemin yapıldığı bir bağlam (Context) vardır ve onun yapılacak işlerle ilgili bir arayüzü bulunur. Context, bir strateji arayüzü ya da soyut sınıfı üzerinden somut strateji nesnelerini bilir; her somut strateji, algoritma arayüzündeki işi kendine göre farklı biçimde kodlar.

Kalıp yapısı (ekrandan, 00:24:30):



Slayttaki bileşenler şunlardır:

- `Strategy`: desteklenecek bütün algoritmaların ortak arayüzünü belirler.
- `ConcreteStrategyX`: algoritma gerçeklemeleridir.
- `Context`: algoritmaya ihtiyaç duyan nesnedir.
  - Bir ya da birkaç `ConcreteStrategy` nesnesi ile ilişkilendirilir.
  - `ContextInterface()`: gerekli hallerde algoritma gerçeklemesine durum bilgisini açan metot ya da metotlardır.

1.4 Strateji geçişini kim yönetir?

Hoca önceki şemada Context'te strateji türünden tek bir üye alan olduğunu hatırlatıp önemli bir soru sordu: herhangi bir anda bir stratejiden diğerine geçişi kim yönetecek? Genelde bundan sorumlu ayrı bir varlık olabilir; ayrı bir fabrika metodu kullanılabilir ya da Factory Method örneğinde olduğu gibi bir yapılandırma dosyasından okunarak ilgili strateji Context'e bir setter ile verilebilir.

Slayttaki gerçekleme ayrıntılarına göre (ekrandan, 00:30:00) Strategy ve Context nesneleri seçilen algoritmayı gerçeklemek için etkileşimde bulunur; Context'i kullanacak istekçi nesneler de Context'e bir `ConcreteStrategy` nesnesi sağlar. Hoca burada bir olasılığı tartıştı: strateji soyut sınıfına en uygun gerçeklemeyi Context'e öneren bir karar verme metodu konabilir. Ama bu pek tercih edilmez, çünkü böyle bir metot soyut sınıfın kendi alt sınıflarının neler olduğunu bilmesini gerektirir.

> **Hocanın vurgusu:** Üst sınıfın alt sınıflardan herhangi bir biçimde haberdar olması kalıtımın özüne tamamen terstir. Temel sınıf, kendisinden yeni alt sınıfların türetilebileceği ve alt sınıflardan bağımsız bir sınıftır; bağımlılık alt sınıftan üst sınıfa doğrudur, kalıtım okunun yönü de bunu gösterir. Üst sınıfa alt sınıfları bildiren metotlar eklemek, yukarıdan aşağıya bağımlılıklar kurmak doğru değildir.

Buna karşılık Context'in olası stratejilerin neler olduğunu bilmesinde bir sakınca yoktur; en uygun stratejiye Context kendisi de karar verebilir ve somut stratejilerden biriyle çalışmayı seçebilir. Hoca bir düzeltme de yaptı: Context'in her işi "hangi stratejiyle çalışıyorsak ona ver" diye yürütmesi gerekmez. Somut strateji de Context ile etkileşip ek durum bilgisine ulaşmak isteyebilir. Bu yüzden şemadaki strateji ilişkisi parça-bütün olarak çizilmiştir; parçalar ait oldukları bütünü bilir ve onunla iletişim içindedir. Hoca okun bu biçimde, iki yönlü bir iletişim olarak okunmasını istedi. Context'i kullanacak istekçi nesneler genelde Context'e en uygun somut strateji nesnesini verir. Özetle hangi somut stratejinin kullanılacağına Context de istekçi (client) de karar verebilir; hangisi işinize geliyorsa.

1.5 Kullanım ve zayıf yönler

Kalıp, bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda ve algoritmanın çalışma zamanında değiştirilebilmesi istendiğinde kullanılır. Ayrıca istekçilerin kullanılacak algoritmanın gerçeklenme ayrıntılarını bilmesi gerekmez; çeşitli stratejiler olduğunu ve birinin Context'e verilmesi gerektiğini bilmeleri yeter (ekrandan, 00:30:00).

Kalıbın iki zayıf yönü vardır. Birincisi, istekçiler (ya da Context) farklı stratejilerin varlığından haberdar olmak zorundadır, çünkü duruma göre en uygun stratejiyi seçmeleri gerekir. İkincisi, Context kendi durum bilgisini public üyeler üzerinden ister istemez stratejiye açar; ama her stratejinin Context'in durum bilgisinin tamamına ihtiyacı olmayabilir. Context ile strateji farklı adres uzaylarındaysa, örneğin farklı makinelerdeyse ve aradaki haberleşme gecikmesi yüksekse, bu, durum bilgisinin tamamına ihtiyaç duymayan stratejiler için ek haberleşme yükü demektir. Hoca bunun her strateji için geçerli olmadığından çok ciddi bir zayıflık sayılmayacağını söyledi.

1.6 Builder ile karşılaştırma

Hoca, Strategy'nin kalıtım ve çok biçimliliğin gücünü kullanan bir kalıp olduğunu ve bu özelliği taşıyan başka kalıplar da bulunduğunu söyleyip Builder'ı hatırlattı. Builder'da bir Director vardı; Director soyut bir builder üzerinden onun somut alt sınıflarından biriyle ilişkilendirilir, bir eleman listesindeki her elemanı alıp o elemanın gerektirdiği işlemi sırayla yaptırırdı; işi aslında somut alt sınıflardan biri yapıyordu.

Sınıf yapısı açısından iki kalıp arasında fark yoktur: RTF okuyucu ile uygulama, `TextConverter` ile satır bölmeleyici, alttaki çeşitli dönüştürücüler ile alttaki çeşitli bölmeleyiciler birbirine karşılık gelir. Fark ayrıntıdadır. Builder'da Director bütün alt işlerin neler olduğunu bilir ve her alt işi ilgili alt sınıfa tek tek yaptırır: "şu parçayı oluştur, sonra bu parçayı oluştur". Strategy'de ise uygulama "yapılacak bir iş var, ilgili durum bilgim de şu; al bu işi yap" der. Yani Builder işi adım adım yaptırırken Strategy komple yaptırır.

> **Hocanın vurgusu:** İki tasarım kalıbının sınıf şemaları birbirine çok benzeyebilir, hatta aynı olabilir. Onları ayıran şey, arka planda işin nasıl yapıldığına ilişkin ayrıntılardır; bu farklara iyi hâkim olmak gerekir.

2. Strategy Örneği: Uluslararası Kişisel Finans Programı

2.1 Problem

Örnek "Design Patterns Explained" kitabından alındı (ekrandan, 00:37:10). Uluslararası pazara hitap eden bir kişisel finans programı yazılıyor; program farklı ülkelerde kullanılabilmeli, yani değişik ülkelerin değişik vergi kurallarıyla başa çıkabilmelidir. Ana programın adı `TaskController`'dır. Hoca, vergi kurallarının ayrıntısına dalıp kalıbın ruhundan uzaklaşmamak için senaryoyu basit tuttuğunu söyledi: bütün gelir sabit bir oranla vergilendirilir. Başlangıçta üç pazar destekleniyor: Türkiye, ABD ve Avrupa Birliği; sonra yeni ülkelere geçilebilir.

2.2 Birinci çözüm: switch-case

İlk akla gelen, bir enum koyup switch-case kullanmaktır. `TaskController`'da enum türünden bir `code` üye alanı ve onun setter'ı vardır. Bütün finansal uygulamalarda ortak bir işlem olarak yıllık gelir verilince vergi tutarını hesaplayan `taxAmount` metodu bulunur.

Sınıf şeması ve kod (ekrandan, 00:37:10):

```
<<Enum>> Country            TaskController
TR, US, EU      <- - - -    code: Country
                            + taxAmount(income: double): double
                            + setCountry(code: Country)
```

```
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code){
        this.code = code;
    }
    public double taxAmount(double income){
        switch (code) {
            case TR: return income * 0.18;
            case EU: return income * 0.15;
            case US: return income * 0.12;
            default: return income * 0.10;
        } } }
```

Hocanın anlatımıyla: ülke Türkiye ise gelirin %18'i, Avrupa Birliği ise %15'i, ABD ise %12'si vergi olarak hesaplanır; bunların dışındaki her ülke için varsayılan oran %10'dur.

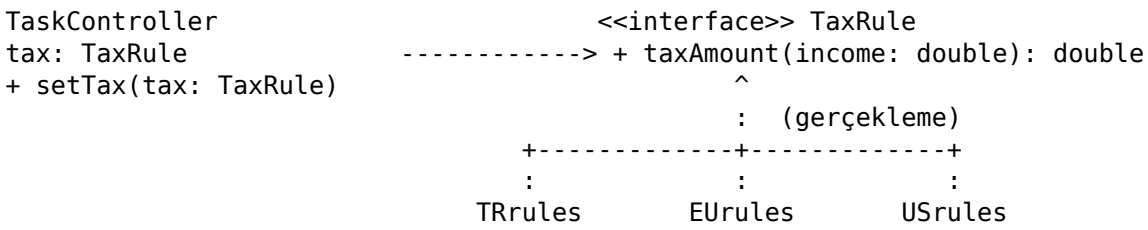
> **Hocanın vurgusu:** Bir yerde kendinizi switch-case kullanırken görüyorsanız, bu aslında kalıtım ve çok biçimlilik kullanmanız gerektiğinin ifadesidir.

Bu gerçek bir uygulama olsaydı her case'in içinde yüzlerce satır olurdu ve kocaman, karmaşık, uzun bir `taxAmount` metodu ortaya çıkardı. Bütün bakım işlemleri pahalılaşır: bir hatayı düzeltmek için taranması gereken kod çok fazladır; ilgili case bulunsa bile içi çok ayrıntılıdır. Yeni bir ülke eklemek istendiğinde (hoca Avrupa Birliği'nden ayrılan Birleşik Krallık'ı örnek verdi) enum'a yeni bir değer, switch'e de yeni bir case eklenir; metot her seferinde bir o kadar daha karmaşık ve uzun olur.

2.3 İkinci çözüm: Strategy kalıbı

Vergilerin farklı ülkelerde farklı biçimlerde hesaplandığını fark edince, belli bir işin farklı yöntemlerle yapılmasını destekleyen kalıbın Strategy olduğunu anlarız. `TaskController`'a artık enum konmaz; vergilendirmeyle ilgili işlemler bir arayüzde tanımlanır ve Türkiye kuralları, Avrupa Birliği kuralları, ABD kuralları gibi sınıflar bu arayüzü gerçekler. `TaskController` bu arayüz türünden bir üye alan tanımlar; `setTax` metoduyla hangi bölgede çalışılacaksa onun kuralı verilir. Vergi hesaplanacağı zaman `TaskController` bu üyenin `taxAmount` metodunu çağırır.

Sınıf şeması (ekrandan, 00:40:00):



Böylece yeni bir ülke geldiğinde yapılması gereken tek şey bu arayüzü gerçekleyen yeni bir sınıf oluşturmaktır; başka hiçbir yerde değişiklik gerekmez. Kod "parçala ve yönet" mantığıyla küçük, yönetilebilir parçalara ayrılmıştır; yalnız Türkiye'nin vergi kuralı değişirse yalnız o sınıf düzenlenir. Öteki türlü ilgili case'i bulup onun içine girmek daha zordur. Hoca bu basit örnekte farkın büyük görünmeyebileceğini, ama gerçek hayattaki gibi çok daha karmaşık vergi kurallarında Strategy'nin yararının açıkça görüleceğini söyledi. Kodun kendisi basittir, ekstra bir numara yoktur. 2. çözümde `TRrules` sınıfı şöyledir (ekrandan, 00:45:10):

```java
package dp.strategy.solution2;

public class TRrules implements TaxRule {

    public double taxAmount(double income) {
        return income * 0.18;
    }

}
```

2.4 Yeni işler eklenirse

Hoca, vergilendirme dışında başka işlerin de olabileceğini sordu: vergi indirimine başvurma, asgari geçim indiriminin hesaba katılması, aile desteğine başvurma ya da kredi borcu hesaplama gibi. Bu işler de ülkeden ülkeye farklıdır. İki seçenek vardır:

- Her işlem için Strategy kalıbı ayrı ayrı tekrarlanır. Örneğin aile desteği için ayrı bir üye alan ve ayrı bir arayüz tanımlanır, o arayüzün somut gerçeklemeleri yazılır. Bu, daha ince ayarlı bir "parçala" yaklaşımıdır.
- Yeni işler ilk strateji arayüzüne yeni metotlar olarak eklenir. Belirli bir ülkeyle ilgili bütün işlemlerin tek bir sınıfta toplanması tercih ediliyorsa bu yol seçilir.

Hocaya göre ikisi de yapılabilir; hangisinin seçileceği tasarımcının tercihidir.

2.5 Stratejiler Singleton adayıdır

> **Hocanın vurgusu:** Strategy kalıbındaki strateji gerçeklemeleri iyi birer Singleton adayıdır. Türkiye kurallarından tek bir tane olacaktır, başka sürümü olmayacaktır; Avrupa Birliği kuralları için de aynısı geçerlidir. Şart değildir, ama aklınızda olsun.

Hoca Eclipse'teki kodu ve Singleton slaytını yan yana açıp farkı gösterdi (ekrandan, 00:45:10). Singleton yapmak için her somut strateji sınıfında kendi türünden statik bir `instance` üyesi, `private` bir kurucu (gerekli ilklendirmeler orada yapılır) ve statik bir `getInstance` metodu kodlanmalıdır; `getInstance`, `instance` ilklendirilmemişse ilklendirir ve her durumda onu döndürür:

```java
package singleton.example;
public class TRrules implements TaxRule {
    private static TRrules instance;
    private TRrules( ) {
        //gerekli ilklendirmeler
    }
    public static TRrules getInstance() {
        if( instance == null )
            instance = new TRrules( );
        return instance;
    }
```

```
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

Slayta göre `EUrules` ve `USrules` de benzer biçimde gerçeklenir. `TaskController` artık işi bu tek örneğe yaptırır. Bu ek karmaşıklığa değip değmeyeceği duruma göre değerlendirilir. Strateji gerçeklemelerinin çalışma ayrıntıları çeşitli setter metotlarıyla verilen bilgilerle değişebiliyorsa durum farklılaşır. Örneğin program farklı şirketlerin vergisini hesaplıyor ve bir şirket için `TRrules`'un bir örneği, öbür şirket için başka bir örneği farklı parametrelerle çalışıyorsa, bu genelde istenmez. O zaman "Türkiye kuralı tektir, Türkiye'deki şirketlerin vergisi tek bir ortak kural üzerinden hesaplanır" denir ve bu eş biçimliliği sağlamak için Singleton kurmanın ek çabası gösterilir.

3. State Kalıbı

Aradan sonra hoca State kalıbına geçti ve bu kalıbın şema olarak Strategy'ye benzediğini, ikisini karşılaştırarak farklarını vurgulayacağını söyledi.

3.1 Amaç

State, adından da anlaşıldığı gibi bir nesnenin, bir varlığın durumuyla ilgilidir. Amaç, nesnenin iç durumu değiştiğinde davranışının da değişmesini sağlamaktır. Hoca burada küçük ölçekli bir davranış değişikliğinden söz etmediğini vurguladı: herhangi bir sınıfın üye alanlarını farklı değerlere ayarladığınızda metotlarının farklı sonuçlar vermesi zaten en temel davranıştır. Burada kastedilen, nesnenin çalışma biçimini tamamen ya da çok büyük oranda değiştiren büyük ölçekli, bütüncül bir değişikliktir.

3.2 Dörtlü Çete'nin örneği: TCP bağlantısı

Bir haberleşme çerçevesi (framework) yazıyoruz ve orada TCP'yi gerçekliyoruz. Bir TCP bağlantısı herhangi bir anda üç durumdan birinde olabilir: Established, Listen ve Closed. Bağlantı kapatılabilir ya da dinleme durumuna geçirilebilir. Dinlerken bağlantı farklı davranır, farklı sinyallere farklı yanıtlar verir. Kapandığı andan itibaren dinleme işlemi yapamaz, ancak yeniden kurulmuş (established) duruma geçirilebilir; örneğin dinleme komutlarına yanıt vermez.

Bu bütüncül değişiklik switch-case ile de kodlanabilir; ama hoca bunun esnek olmadığını ve ne gibi sakıncalara yol açtığını önceki kalıplarda birçok kez gördüğümüzü söyleyip switch-case çözümüne bakmadı bile. İlke yine "neyin değiştiğini bul ve onu sarmala"dır. Değişen şey TCP soketinin durumlarıdır; bu durumlar sarmalanır, kalıtım doğru kullanılır ve bir parça-bütün ilişkisi kurulur.

Dörtlü Çete'nin önerisinde bir `TCPConnection`'ın bir durumu (state) vardır. Bu üye, `TCPState` türünden bir soyut sınıf ya da arayüz olabilir. `TCPConnection` üzerinde açma, kapama ve alındı verme (acknowledge) komutları yürütülebildiği için aynı metotlar durum arayüzünde de bulunur. `TCPConnection`, TCP işlemleriyle ilgili (`Open`, `Close`, `Acknowledge`) işleri kendisi yapmaz, o andaki duruma yaptırır. Üç durum ayrı birer alt sınıf olarak gerçeklenir: `TCPEstablished`, `TCPListen` ve `TCPClosed`.

3.3 Kalıp yapısı

Kalıbın genel yapısında yine bir Context vardır; ama Context'in içinde bulunabileceği durumlar da vardır ve bu durumlar `State` arayüzüyle modellenir. Context'e herhangi bir istek (request) geldiğinde işin asıl karşılanması (handle) o andaki durum üzerinden yapılır.

Kalıp yapısı (ekrandan, 01:04:50):

```
Context <>------- state -------> State
  Request()                        Handle()
  // state->Handle()                 ^
                              +--------+--------+-  -  -
                              |                 |
```

```
ConcreteStateA     ConcreteStateB
  Handle()           Handle()
```

TCP örneğinde istek metodu ile karşılama metodunun adı aynıydı; ama bu zorunlu değildir, `Handle` metodunun adı farklı olabilir.

3.4 Durum geçişlerinden kim sorumlu?

Hoca, varlığın bir durumdan başka bir duruma nasıl, ne zaman ve hangi kurallar altında geçeceğinden genellikle Context'in sorumlu olduğunu söyledi; çünkü Context kendi durum bilgisini en iyi bilen, bütün durum bilgisine sahip olan nesnedir. Aksi halde bundan durumlar sorumlu olabilir; bu da yapılabilir. Soyut `State` sınıfının alt sınıflarını bilmesine gerek yoktur. Context-State ilişkisi iki yönlü kurulmuşsa (parça durum, bütün Context), parçalar ait oldukları bütünü bilebildiğinden gerekirse Context'i sorgulayarak ek bilgi alabilir.

Hoca ardından her durumun, kendisinden hangi koşullarda hangi durumlara geçilebileceğini daha çok bildiğini de ekledi. Bu bilgi somut durumlarda kullanılabilir: belirli bir somut durumdayken belirli bir şart altında belirli bir olay gerçekleşince, somut durum Context'i yeni duruma geçirebilir. Ya da Context kendi durum bilgisine bakarak "ben bu durumdaydım, şu olay gerçekleşti, şu duruma geçeceğim" diyebilir. İkisi de kabul edilir, ama hocaya göre genelde kararı Context verir. Slayt da (ekrandan, 01:10:00) durum geçişlerinin açıkça, Context'e verilen komutlarla ya da `ConcreteState`'lerin içinden yapıldığını söyler.

3.5 Ne zaman kullanılır?

Slayta göre (ekrandan, 01:10:00) kalıp şu durumlarda kullanılır:

- Bir nesnenin çalışma şeklinin, çalışma anında o nesnenin durumuna göre değişebileceği yerlerde.
- Bir nesnenin durumuna bağlı, karmaşık ve uzun, çok kısımlı karar verme komutlarının verilmesi gereken anlarda.

3.6 State ve Strategy farkı

Hoca iki kalıbın farkını üç maddede topladı:

- En temel fark: Strategy'de strateji geçişleri genelde seyrektir, State'te durum değişiklikleri daha sıktır. Hoca bunu sözcüğün anlamıyla açıkladı: bir problemi çözerken ya da bir savaşta strateji, uzun vadeli oyun planınızdır; stratejiler kısa dönemde değişmez. Tasarım kalıbında da bir algoritma seçilir ve uzunca bir süre onunla devam edilir. Oysa bir varlık bir durumdan başka bir duruma çok rahat geçebilir.
- İkinci fark: strateji nesneleri daha çok kendi metotlarına verilen parametreler üzerinden iş yapar; Context'i sorgulayabilseler de buna daha az ihtiyaç duyarlar. Durum nesneleri Context'i daha sık sorgular, çünkü bir durum, varlığın üye alanlarının değerlerine çok bağımlıdır ve herhangi bir değişiklik durumları ve geçişleri daha çok etkiler.
- Üçüncü fark: State'te genelde Context kendi somut durumunu seçer; bu da State'in genel ayırt edici noktasıdır.

Slayttaki özet (ekrandan, 01:10:00):

- State:
  - Context'teki durum değiştikçe Context kendi `ConcreteState`'ini seçer.
  - Durum geçişleri açıkça yapılır (Context'e verilen komutlarla ya da `ConcreteState`'lerin içinden).
  - Durum geçişleri daha sıktır.
- Strategy:
  - Strateji geçişleri seyrektir.
  - Strateji nesneleri Context hakkında daha az şey bilir, kendilerine gelen parametrelere göre işlem yapar.

3.7 Sonlu durum makinesi benzetmesi

Hoca derleyici tasarımı gibi dersler almış olanların sonlu durum makinelerini (finite state automata) bildiğini hatırlattı; bunların kimi deterministik, kimi deterministik olmayan otomatalardır. Böyle bir otomatadaki her durum, tasarım kalıbındaki bir duruma denk gelir. Şemada bir durumdan hangi olay gerçekleşince, hangi girdi alınınca başka bir duruma geçileceği ya da aynı durumda kalınacağı bilgisi, o durumu simgeleyen somut durum sınıfının içine kodlanır. Hocaya göre sonlu durum makinelerindeki durumlarla State kalıbındaki durumlar arasında çok yakın bir benzerlik vardır.

4. State Örneği: Kişisel Bilgi Yöneticisi

4.1 Problem

Hoca örnekleri arasında en az hoşuna gidenlerden birinin State örneği olduğunu, daha iyi bir örnek vermek istediğini ama bu dönem buna fırsat bulamadığını söyledi; yine de kalıp yeterince tartışıldığı için basit bir örnek üzerinden gidildi.

Bir kişisel bilgi yöneticisi (Personal Information Manager, PIM) programı yazıyoruz. Program tanıdıklarımızın telefon, e-posta, web sayfası ve sosyal medya adresi gibi erişim bilgilerini tutuyor. Kayıtlar iki durumdan birinde olabilir: temiz (clean) ya da kirli (dirty). Temiz, kaydın güncel olduğu ve değişiklik yapılmadığı durumdur. Kirli ise kayıtta değişiklik yapılmış ama henüz kaydedilmemiş durumdur. Kirli durumdayken pencere kapatılmak istenirse program "değişiklikler yaptınız, kaydetmek istiyor musunuz?" diye sormalıdır. "Evet" denirse kaydedilerek, "hayır" denirse kaydetmeden (değişikliklerden vazgeçilerek) temiz duruma geçilir.

Hoca günlük bir örnek de verdi: Eclipse'te bir sınıfın kodunu değiştirmeye başlayınca dosya adının başında bir yıldız belirir, kaydedince yıldız gider. O yıldız, kaydın değiştirilmiş ve henüz kaydedilmemiş, yani kirli durumda olduğunu gösterir.

> **Hocanın vurgusu:** "Kirli ve temiz durumu" diye ağzımızdan bir kere "durum" kelimesi çıktıysa ve bu kelime yerine oturuyorsa, bu problemi State kalıbıyla tasarlayabiliriz.

4.2 Tasarım

Her kayıt bir durum üyesine sahip olabilirdi. Durum sınıfı soyut bir sınıftır ama içindeki `edit` ve `save` metotları somuttur. Kaydın düzenlenmesi ile kirli ve temiz durumlarına geçiş yalnız bir şekilde, kayıt işlemiyle gerçekleşir. Bu yüzden bu örnekte durum mantığının bir kısmı soyut sınıfta, bir kısmı alt sınıflarda kodlanmıştır. Hoca bunun kalıbın geleneksel yorumundan biraz saptığını, ama gerekirse yapılabilecek bir şey olduğunu söyledi.

Uygulamada temiz ya da kirli durum kayıt bazında değil, bütün uygulama düzeyinde tutulur: ister bir kayıt düzenlenip kaydedilmemiş olsun, ister on kayıt, uygulama kirli sayılır. Uygulama durum üyesi üzerinden çalışsa da durumların neler olduğunu bilmek ve bu durumlar arasında geçiş yapmak zorundadır. Hoca bunun Strategy ile State arasındaki en önemli farkı yeniden gösterdiğini söyledi.

`PIMApplication` sınıfında (ekrandan, 01:20:00) olası durumlar `public final static` üyeler olarak tanımlanmıştır; ikisi de `State` türündendir. O anki durum ise elbette `private`'tır:

```
PIMApplication
+ CLEAN: State
+ DIRTY: State
- currentState: State
+ getCurrentState(): State
+ setCurrentState(state: State)
```

Temiz durumdayken düzenleme yapılırsa durum, uygulamayı kirli duruma geçirmek zorundadır; kaydedildiğinde ise temiz duruma geçilir.

4.3 Kod

Eclipse'teki `dp.state.example` paketinde `CleanState`, `DirtyState`, `PIMApplication`, `Record` ve `State` sınıfları vardır (ekrandan, 01:20:00). Hoca kaydetme işini `State` soyut sınıfında kodladığını söyledi: kayıt temizken de kirliyken de yapılsa nihayetinde aynı kayıt işlemidir, bu yüzden bu ortak davranış üst sınıfta durur. `edit` metodunun ise aslında soyut olması daha iyi olurdu; kod kalitesi uyarıcısı da "soyut bir sınıftaki boş bir metot soyut olmalıdır" diye uyarıyordu ve hoca bunun doğru olduğunu kabul etti. Ama iki durumda da düzenleme temelde aynı işi yapacağı için varsayılan bir davranışın üst sınıfa konması burada mantıklıdır; gerçek uygulamada bu metoda kod yazılınca uyarı da kalkacaktır.

`CleanState` (ekrandan, 01:17:50):

```
package dp.state.example;
import java.io.*;
public class CleanState extends State {
    public void edit( ){
        PIMApplication.setCurrentState(PIMApplication. ... );
        super.edit();
    }
    public void save(File f, Serializable s) throws IO... {
        super.save(f, s);
        PIMApplication.setCurrentState(PIMApplication. ... );
    }
}
```

Temiz durumdayken düzenleme isteği gelince işin kendisi yine üst sınıfın `edit` metoduyla yapılır; ama önce uygulamanın o anki durumu kirliye geçirilir. `save` ise kaydı yürüttükten sonra durumu temize geçirir. Böylece bir durum, hangi koşulda hangi yeni duruma geçileceği ya da aynı durumda kalınacağı bilgisini kendi kodunda tutar ve geçişi kendisi yürütür. Hoca bu örnekte gerçeklemeyi bu yönden tercih ettiklerini söyledi.

`DirtyState` (ekrandan, 01:20:00):

```
package dp.state.example;
public class DirtyState extends State { }
```

Kirli durumda yapılacak ek bir iş yoktur; kirliyken düzenleme yapılınca kalıtımla gelen üst sınıfın `edit`'i çalışır. Hoca, bir üst sınıftan türeyip kendisine yeni bir metot ya da üye alan eklemeyen böyle sınıflara "marker (işaretçi) sınıf" dendiğini söyledi; bunlar bir ayar yapıldığını gösteren sınıflardır. Java'da serileştirme (serialization) çalışmış olanlar bunu bilir: serileştirilecek sınıfların `Serializable` arayüzünü gerçeklemesi gerekir ve bu arayüzde hiçbir metot tanımlı değildir; yalnız "bu nesne serileştirilebilir" bilgisini işaretler.

> **Ders sonrası not:** Hoca serileştirmeyi nesnenin durum bilgisinin sınıfın kaynak ya da bayt koduyla birlikte dosyaya yazılması diye anlattı. Java serileştirmesi büyük olasılıkla nesnenin alan değerlerini ve sınıfı tanıtan bir betimleyiciyi (sınıf adı, `serialVersionUID`) yazar, sınıfın bayt kodunu yazmaz; okuma sırasında sınıfın kendisi ayrıca yüklenmiş olmalıdır.
>
> **Ek bilgi (derste anlatılmadı):** Java'da metot içermeyen ve yalnız bir yeteneği işaretleyen arayüzlere "marker interface" denir; `Serializable` ve `Cloneable` en bilinen örnekleridir. `ObjectOutputStream`, yazacağı nesnenin `Serializable` olup olmadığını denetler, değilse `NotSerializableException` fırlatır.

`PIMApplication`'da `CLEAN` ve `DIRTY` statik üyeleri ilklendirilir ve başlangıç durumu temizdir. Düzenleme ve kaydetme işleri geldiğinde uygulama bunları kendisi yapmaz, o anki duruma yaptırır; kalıp da bunu ister.

> **Hocanın vurgusu:** Olay ayrıntıda gizlidir. Durumlar, kendi durumlarından aynı durumda kalmanın ya da başka bir duruma geçmenin hangi koşullarda olacağını kendi kodlarında yazar ve gerekli geçişlerin yürütülmesini sağlar.

5. Template Method Kalıbı

5.1 Amaç

Strategy'de algoritma komple değişiyordu. Peki alternatif algoritmalar arasında ortak yürüyecek bazı adımlar varsa ne yapılır? Template Method bu soruya yanıt veren tasarım kalıbıdır. Amaç, bir algoritmanın ortak ve değiştirilmemesini istediğimiz bazı adımlarını belirleyip geri kalan adımların ayrıntılarının belirlenmesini ertelemektir. Bu ayrıntılar alt sınıflarda belirlenecektir.

5.2 Dörtlü Çete'nin örneği: belge çerçevesi

Farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve (framework) hazırlıyoruz. Çerçeve, belge ve uygulama sınıflarından oluşur. Çerçevenin kullanıcıları bu sınıfların alt sınıflarını oluşturarak kendi belge türleri üzerinde çalışan kendi uygulama türlerini yaratacaktır. Her belge türünde yapılabilecek ortak işlemler vardır; bunları mümkün olduğunca çerçevede gerçekleyip yükü kullanıcıdan almak istiyoruz. Hocaya göre çerçevenin amacı da budur: bazı işleri hazır kodlayıp onların yükünü kullanıcıdan almak. Temel ve ortak adımların doğru sırayla yürütülmesi çerçevede kodlanır, diğer adımları uygulama yapar.

Sınıf şeması ve kod (ekrandan, 01:30:00):

```
Document  <-------- docs -------<> Application
  Save()                            AddDocument()
  Open()                            OpenDocument()
  Close()                           DoCreateDocument()     (soyut)
  DoRead()      (soyut)             CanOpenDocument()      (soyut)
     ^                              AboutToOpenDocument()  (soyut)
     |                                   ^
     |                                   |
MyDocument  <- - - - - - - - - - -  MyApplication
  DoRead()                            DoCreateDocument()   // return new MyDocument
                                      CanOpenDocument()
                                      AboutToOpenDocument()
```

```
void Application::OpenDocument (const char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        _docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

Şemada çizginin üst tarafı çerçeve, alt tarafı çerçeve kullanılarak gerçeklenen uygulamadır. Her belge mutlaka kaydedilecek, açılacak ve kapatılacaktır; dosyaya bilgi aktarılması, dosyanın fiziksel olarak açılması ve kapatılması belge türünden bağımsız kodlanabilir. Ama okumanın kendisi farklı uygulamalarda, farklı belge türlerinde farklı yapılacaktır. Bu yüzden çerçevede `Document` sınıfı oluşturulur, `Save`, `Open` ve `Close` metotları çerçevede kodlanır, `DoRead` ise soyut bırakılıp alt sınıfta tanımlanır. Çerçevenin kullanıcısı kendi belge türünün nasıl okunacağını bu alt sınıfta kodlar, gerisini çerçevenin `Document` sınıfına bırakır.

`Application` da farklı türden belgeler olsa bile açık belgelere yeni bir belge eklemek için `AddDocument`, bir belgenin açılmasını başlatmak için `OpenDocument` metoduna sahiptir. Belge için gerekli ilk içeriğin oluşturulması (`DoCreateDocument`), bu uygulamanın bu tip bir belgeyi açıp açamayacağına karar verilmesi (`CanOpenDocument`) ve belgeyi açmadan önce gereken hazırlıkların yapılması (`AboutToOpenDocument`) ise soyuttur, çünkü farklı uygulamalarda farklı biçimde

yapılmaları gerekir. Ortak kısımlar üst sınıfta kodlanır; farklı yapılması gerekenleri çerçevenin kullanıcısı `Application`'ın bir alt sınıfını oluşturarak kodlar.

Dikkat edilmesi gereken kod bloğu `Application`'ın `OpenDocument` metodudur. Bu somut bir metottur ama içinde soyut metotlardan yararlanır. Bir belge açılmadan önce açılıp açılamayacağı denetlenmelidir; bu yüzden `CanOpenDocument` çağrılır. Bu metot soyuttur, yani `Application`'ın kendisinde değil alt sınıfında tanımlanmıştır. `OpenDocument` kalıtımla `MyApplication`'a geçtiğinde, orada somut olarak tanımlanmış `CanOpenDocument` çalışır ve açılmak istenen belgenin bu uygulama tarafından açılıp açılamayacağı değerlendirilir. Açılamıyorsa metot döner; belki "desteklenmiyor" diye bir iletişim kutusu açılır. Açılabiliyorsa belge oluşturulur; gerçekten oluşturma işi de uygulamaya bağımlı olduğundan alt sınıfın metodu çağrılır. Sonra belge, belgeler listesine eklenir; açılmadan önce yapılması gereken işler varsa alt sınıfa bunları yapma şansı tanınır; ardından belge fiziksel olarak açılır ve okuma başlar.

> **Hocanın vurgusu:** İşin bütün esprisi şudur: `OpenDocument` metodunun hem sabit hem değişebilen adımları vardır. Sabit adımlar burada kodlanır, değişebilen adımlar ise soyut metotlar üzerinden çağrılır; bu metotlar alt sınıfta somut olarak tanımlanmış olur.

5.3 Şablon metot final olmalı

Hoca bir ayrıntı daha ekledi: `OpenDocument` metodunun alt sınıfta override edilmesi istenmez. Bu metotta neyin hangi sırayla yapılacağı özenle kodlanmıştır; biri acemilikten ya da kötü amaçla bu metodu override edip uygulamanın hiç olmaması gerektiği gibi davranmasına neden olabilir. Bu yüzden metodu `final` olarak işaretlemekte yarar vardır; `final` metotlar alt sınıflarda override edilemez. Hoca, slayttaki bu notun Java ayrıntısını kendi Java bilgisiyle eklediğini, çünkü Dörtlü Çete kitabı yazılırken Java dilinin henüz olmadığını, C++ ile ilgili kısmı ise kitaptan olduğu gibi aldığını söyledi: C++'ta alt sınıfta mutlaka tanımlanması gereken metotlar saf sanal (pure virtual) olarak işaretlenir.

5.4 Kalıp yapısı

Kalıp yapısına bakıldığında bir soyut üst sınıf ve bir somut alt sınıf vardır. Soyut üst sınıfta, kalıba adını veren şablon metot (template method) bulunur. Şablon metodun içinde ayrıntıları değişecek soyut metotlar çağrılır; bu değişebilen işlemlere "primitive operation" denir ve bunların nasıl yapılacağı alt sınıfta kodlanır. Şablon metodun içindeki diğer yerler sabit adımlardır. Yapı bu biçimde kurulduğunda her şey yolunda gider.

5.5 Ne zaman kullanılır?

- Algoritmanın değişmez kısımları bir kez kodlanıp değişebilecek kısımları alt sınıflarda gerçeklenmek istendiğinde.
- Alt sınıfların bazı davranışlarını denetlemek istendiğinde. Hoca bunun çok önemli bir kullanım senaryosu olduğunu söyledi. Normalde bir üst sınıftan istediğiniz gibi alt sınıf türetebilir, metotlarını istediğiniz gibi override edebilir ve yeni metotlar ekleyebilirsiniz; olması gereken de budur. Ama bunun olmaması gereken anlar da olabilir. Bazı işlemlerde alt sınıflara yalnız sınırlı bir esneklik tanımak istiyorsak denetlemek istediğimiz metodu üst sınıfta `final` tanımlarız; sabit kalacak yerleri bu metotta kodlar, değişebilen yerlerin soyut metotlar üzerinden alt sınıflarda gerçeklenmesine izin veririz.
- Kardeş sınıflarda ortak yapılan işleri ortak bir üst sınıfa çekmek. Bu aslında kalıtımın genel bir amacıdır, ama Template Method'da daha da belirgindir.

6. Template Method Örneği: Veritabanından Bağımsız Sorgu

6.1 Problem

Bu örnek de "Design Patterns Explained" kitabından alındı. Yazacağımız uygulama veritabanından kayıt çekecek; verinin kalıcılığını sağlamak gerekiyor. Genelde bir SQL komutu, bir SELECT komutu inşa edilir ve çalıştırılır. Bu komutun yapılandırılması farklı veritabanı yönetim sistemlerinde farklı olabilir; biz ise uygulamamızın veritabanından bağımsız olmasını,

farklı VTYS'lerin herhangi biriyle çalışabilmesini istiyoruz. Özellikle saklı yordam (stored procedure) dilleri VTYS'ler arasında çok daha fazla farklılık gösterir. Her VTYS için uygulamanın ayrı bir sürümünü mü çıkaracağız? Hayır.

6.2 Neden Strategy değil?

Burada ilk akla gelen Strategy olabilir: farklı veritabanlarına erişim için farklı strateji gerçeklemeleri yazılabilir. Ama Strategy'de işin hepsini stratejiler yapar. Bu örnekte farklı veritabanları için yürütülecek farklı işler olduğu kadar ortak işlemler de vardır.

> **Hocanın vurgusu:** Anahtar kelime şudur: yürütülecek ortak işlemler de varsa Template Method kalıbı kurulur.

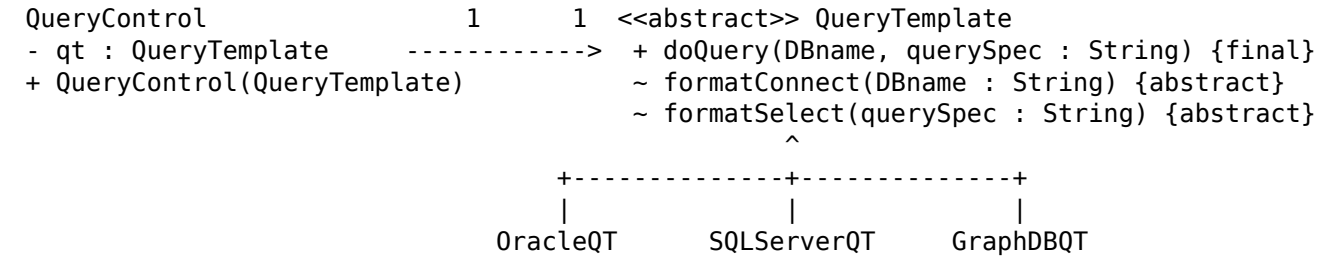
Bütün veritabanı farklılıklarına rağmen iş beş temel adımda yürür:

- Bir bağlantı komutu (connection string) biçimlendirilir.
- Bu komut kullanılarak veritabanına bağlanılır.
- Bir SELECT komutu inşa edilir.
- Komut veritabanına gönderilir.
- Veritabanı sonuç kümesini geri döndürür.

Bu beş adım veritabanından bağımsız olarak yürür. Ortak adımların bulunması Template Method'u burada daha doğru bir seçim yapar.

6.3 Önerilen çözüm

Sınıf şeması (ekrandan, 01:36:10):



Slayttaki notlar:

- Komut biçimlendirme metotları iç çalışma ile ilgili görülerek protected tanımlanmıştır.
- Bir sınıf kütüphanesi ya da çerçeve program hazırlanmadığına göre Java için paket görünürlüğü kullanılabilir.

Sorgu, veritabanı adına ve bir sorgu belirtimine (query specification) göre yapılır. `doQuery` `final`'dır; şablon metot budur. `formatConnect` ve `formatSelect` soyuttur. Hoca bu soyut metotları paket görünürlüğünde (şemadaki ~) tanımlamayı tercih ettiğini söyledi: dış paketlerden bu metotlara doğrudan erişilemesin, yalnız alt sınıflar erişebilsin diye. Asıl işi `doQuery` yapar; biçimlendirme metotları yalnız veritabanına özgü kısımları yürütür ve bir işin yalnız veritabanına özgü kısmını tek başına yürütmenin bir anlamı yoktur, bu yüzden dışarıya açılmamıştır. Neden `private` değil? Bu işlerle uğraşacak ve birlikte çalışması gereken başka sınıflar da olabilir; onlar da bu pakette tanımlanır ve gerekirse bu metotları çağırabilir.

Soyut üst sınıftaki somut `doQuery` metodunda önce veritabanı adından yola çıkılarak bağlantı dizesi, alttaki soyut metot üzerinden inşa edilir; sonra select komutu biçimlendirilir; ardından komut yürütülür ve veri kümesi üzerinde işlemler yapılır. Hoca bu son kısmı kodda yalnız yorum olarak bıraktığını söyledi. Örneğin Oracle ile çalışacak sürümün `formatConnect` metodu Oracle'a özel biçimlendirmeyi yapıp sonucu döndürür, `formatSelect`'i de Oracle'a göre seçim dizesini inşa eder. SQL Server sürümü de aynı işleri SQL Server'a göre yapar. Ana uygulamada (`QueryControl`) `QueryTemplate` türünden bir alan vardır ve bu şablon kurucuda verilir. Örneğin bütün kayıtları getirmek gibi bir iş gerektiğinde bu işler `doQuery` üzerinden yürütülür.

7. Visitor Kalıbı

İkinci aradan sonra hoca Visitor'a geçti. Mediator'a geçip geçmeyeceklerine bakacaklarını, Interpreter kalıbının ise slaytlarında zaten olmadığını söyledi.

7.1 Amaç ve sorun

Hocaya göre Visitor, tasarım kalıpları içinde en karmaşık konu sayılabilir: hem anlatılması gereken konunun hacmi hem de yapılan işin karmaşıklığı açısından en ileride olan kalıptır. Parçalardan oluşmuş bir bütünün parçaları ziyaret edilerek onlar üzerinde bir şeyler yapılacaktır.

Slayttaki tanım (ekrandan, 02:00:00):

- Amaç: herhangi bir nesne yapısını oluşturan elemanlar üzerinde yapılacak bazı işlemler tanımlamak.
- Sorun:
  - Nesne yapısı her türden bağıntılar içeren sınıfların örnekleri arasında kurulmuş olabilir.
  - Üzerinde işlem yapılan elemanların sınıf yapılarını değiştirmek istemiyoruz.
- Eleştiri: madem daha yapılacak işlemler var, eylemler ve sorumluluklar tam olarak belirlenmemiş demektir. Dolayısıyla mevcut sınıf yapılarını değiştirerek farklı bir tasarım yapmak daha doğru olabilir.

Hoca Composite kalıbını hatırlattı: orada her düğüm üzerinde değişik işlemler yapılabiliyordu; bir istemci bileşenleri dolaşıp işlem yapabiliyordu. "Daha ne olabilir?" denebilir. Buradaki sorun, nesne yapısının farklı türden bağıntılar içeren nesneler arasında kurulmuş olabilmesi ve üzerinde işlem yapılacak elemanların sınıf yapılarını ekstra metotlarla kalabalıklaştırmak istememizdir; zamanla yeni yeni yapılması gereken işler de çıkabilir.

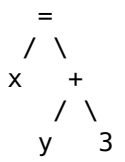
7.2 Eleştiri ve geriye uyumluluk

Hoca slayttaki eleştiriyi açtı: madem daha tanımlanmamış, eklenebilecek ya da değişebilecek işlemler var, belki eylemleri ve sorumlulukları tam belirleyememişizdir; sınıf yapılarını değiştirmek istemesek de değiştirip daha düzgün bir tasarım yapmak daha doğru olabilir. Ama bu her durumda mümkün olmayabilir; mümkünse öyle yapılır. Hoca refactoring'i de ileride göreceklerini söyleyip kısaca tanımladı: refactoring, dışarıya verilen hizmetleri ya da yüzü değiştirmeden iç çalışma yapısını değiştirmektir ve geriye doğru uyumluluğu sağlar. Ama bazen yepyeni bir şey ortaya çıkarmak, iç kalite özellikleri çok daha yüksek bir tasarım ve koda ulaşmamızı sağlayabilir. Her halükârda geriye doğru uyumluluğu korumak, yeni işler ve özellikler çıkınca bunları mevcut yapıyı değiştirmeden sisteme ekleyebilmek istiyoruz. Bu kolay bir iş değildir ve kalıbın karmaşıklığı buradan gelir.

7.3 Örnek: derleyici

Slayttaki örnek (ekrandan, 02:00:00): yazılan bir derleyici kaynak kodu farklı türlerden token'lara ayırır; bu parçalar (node) farklı tiplerdendir; derleyici bütün parçalar üzerinde çeşitli işlemler yapacaktır ve farklı tip parçalarda işlemler farklı yürütülecektir.

Hoca bunu tahtada bir ifadeyle anlattı: `x = y + 3`. Bu komutun ayrıştırma (parse) ağacında bir atama operatörü vardır. Atamanın sol tarafında `x` operandı, sağ tarafında toplamanın sonucu vardır; toplama (`+`) düğümünün de operandları `y` ve `3`'tür.



Derleyici her düğümü tek tek dolaşacak ve farklı işlemler yapacaktır. Örneğin yaprak düğümlerde ilk başta bir işlem yapmaz; ama `+` düğümüne gelince, hele Java'da `+`'nın iki anlamı (dizgi birleştirme ya da aritmetik toplama) olduğu için, sol ve sağ çocuklara bakar ve türlerini alır. Yukarıda bir yerde `x` ve `y`'nin `int` tanımlandığını düşünelim: derleyici `y` ve `3` için tip denetimi

(type checking) yapar, ikisinin de `int` olduğunu görüp +'yı tam sayı toplaması olarak ele alır. Ağacı dolaşırken atama düğümüne gelince yine tip denetimi yapar: soldaki adresin türü ile sağda hesaplanan ifadenin türü aynı mı? Aynıysa derleme hatası vermeden atamayı yürütür. Hoca bu kadar küçük bir örnekte bile ne kadar çok ve ne kadar farklı eylem türü sayabildiğini, gerçek dünyada çok daha karmaşık senaryoların olabileceğini vurguladı.

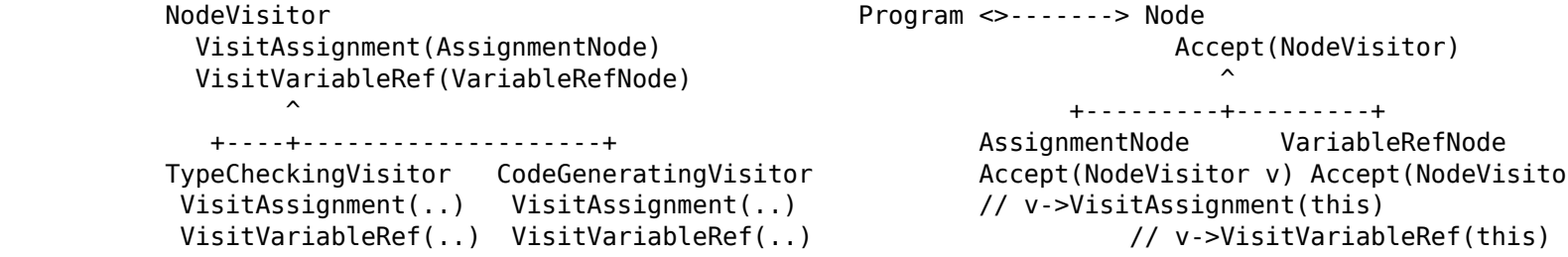
7.4 Önerilen çözüm

İlk yapıda düğümler üzerinde yapılacak işlemler tip denetimi, kod üretimi (code generation) ve düzgün yazım (pretty printing, kodu standarda uygun düzgün yazım haline getirme) olabilir. Farklı türden düğümler de vardır: değişkenlere başvuran düğüm, atama düğümü, aritmetik işlem düğümü ve başka kardeş sınıflar. Hepsinde bu üç işlem yapılacaktır. Bazı düğüm türleri için bazı işlemlerin anlamı olmayabilir; o zaman o metotların gövdeleri boş kalır. Hoca benzer dezavantajı Composite kalıbında da gördüklerini, oradaki sözlerinin burada da geçerli olduğunu söyledi. Ayrıca tip denetimi, kod üretimi gibi işlemler farklı düğüm sınıflarına dağıtılırsa, bir düzeltme ya da değişiklik gerektiğinde neyin nerede yapılacağını bulmak çok zorlaşır; bu tam bir bakım kâbusudur.

Dörtlü Çete'nin önerisi: derleyici parçaları tek tek ziyaret edecek ve her parça üzerinde yalnız gerekli işlemleri yapacaktır. Düğüm gerçeklemeleri kendi üzerlerinde hangi işlemlerin yapılabileceğini bildikleri için derleyiciye "benim üzerimde şu işlemleri yapabilirsin" diye bir kısıtlama da verebilmelidir.

Bunun için mevcut düğüm (ağaç) yapısına `Accept` adlı bir metot eklenir. Parametresi, düğümleri dolaşacak sınıfın türüdür; bu tek bir soyut tür olacaktır. Böylece düğümler yalnız herhangi bir `NodeVisitor` tarafından ziyaret edilebileceklerini bilir. Arka planda `NodeVisitor` soyut sınıfının somut alt sınıfları olur ve her alt sınıf farklı bir eylemin nasıl yapılacağını bütün ayrıntılarıyla içerir. Örneğin `TypeCheckingVisitor` bütün düğümlerde yalnız tip denetimi yapar; ama atama düğümünde ya da değişken başvuru düğümünde bu işi farklı yapabilir. Sonuçta tip denetimiyle ilgili bütün işlemler, hangi düğüm türüyle ilgili olursa olsun, aynı sınıfta toplanmış olur. Önceki durumda bu ayrıntılar ilgili düğüm türlerine dağılmıştı; anlamak, incelemek ve düzeltmek için hepsini dolaşmak gerekiyordu, hatta birinde yapılan düzeltme diğerlerinde de ek düzeltmeler gerektirebiliyordu.

Örnek çözümün şeması (ekrandan, 02:10:00):



Slayttaki açıklamalar:

- Düğüm gerçeklemeleri kendi üzerlerindeki gerekli işlemleri çağıracak ziyaretçi gerçeklemelerini kabul eder (`Accept` metodu ile).
- Bu şekilde parçalarda yalnız gerekli işlemler yürütülür.
- Çözümün zayıf yönü: düğümler ziyaretçinin hangi metodunun bu düğüm türü için olduğunu bilmelidir.

Örneğin `AssignmentNode`'da `Accept` metodu çağrılır; üst sınıfta soyut olan bu metot burada somut olarak tanımlanmıştır. Düğüm ziyaretçinin kendisini ziyaret etmesini sağlar, ama kendisi bir atama düğümü olduğu için "beni bir atama düğümü olarak ziyaret etmelisin" diyerek ziyaretçinin `VisitAssignment` metodunu çağırır. `VariableRefNode` ise `Accept` içinde `VisitVariableRef`'i çağırır ve kendisini verir. Her özelleşmiş düğüm türü, üzerinde dolaşacak ziyaretçiye "ben şu tipte bir düğümüm, beni bu olarak ziyaret etmelisin" der ve ilgili metodu çağırır.

> **Ek bilgi (derste anlatılmadı):** Bu yapıya "çift yönlendirme" (double dispatch) denir. Çalışacak kod iki nesnenin türüne birlikte bağlıdır: `accept` çağrısı çok biçimlilikle elemanın gerçek türüne göre seçilir, eleman da `visitX(this)` çağrısıyla ziyaretçinin gerçek türüne göre doğru metodu seçtirir. Java ve C++ gibi diller metot

seçimini yalnız alıcı nesnenin çalışma anı türüne göre yaptığı için Visitor bu iki adımı elle kurar.

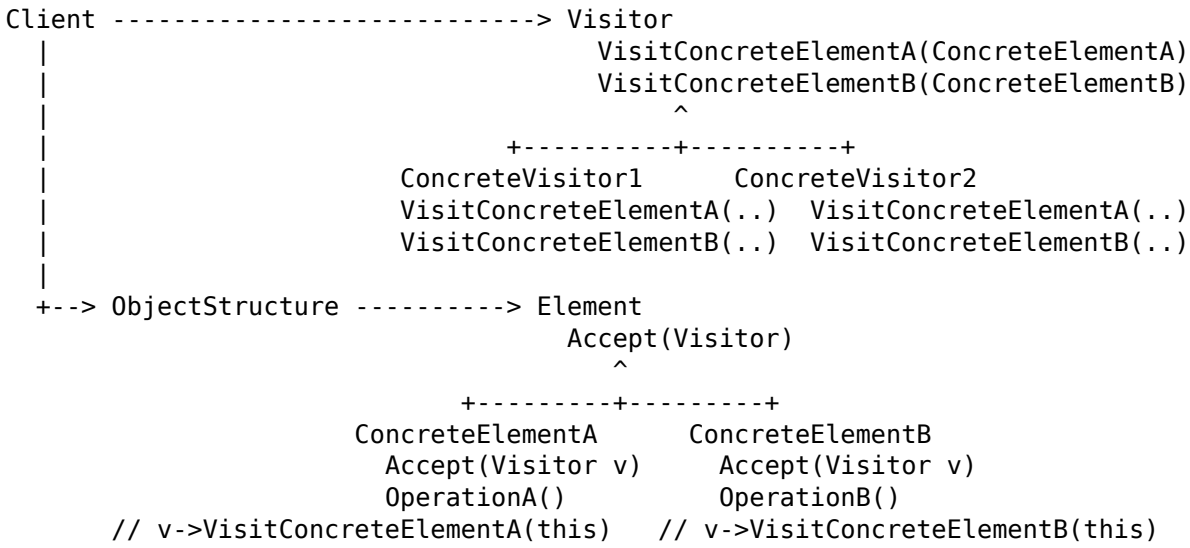
7.5 Zayıf yönler

Kalıbın zayıf yönü, az önce söylendiği gibi, her düğüm türünün ziyaretçinin kendisini hangi metot üzerinden ziyaret etmesi gerektiğini bilmek zorunda olmasıdır: `AssignmentNode`, `VisitAssignment`'ı; `VariableRefNode`, `VisitVariableRef`'i çağırması gerektiğini bilir. Aradaki bu bağımlılık bütün ziyaretçi alt sınıflarına değil, soyut sınıfa yöneliktir. Bu yüzden UML sınıf şemasında her düğüm alt sınıfından `NodeVisitor`'a kesikli çizgiyle gösterilen bir kullanma bağımlılığı vardır; şemayı karmaşıklaştırmamak için çizilmemiştir, ama vardır, ayrıca ziyaretçiler parametre olarak da gelir. Bu ekstra kullanma bağımlılığı ve diğer kalıplara göre nispeten karışık yapı, kalıbın dezavantajlarıdır.

7.6 Kalıp yapısı

Kalıp yapısında üzerinde dolaşılacak elemanlar vardır; bunlar ortak bir soyut arayüz ya da soyut sınıf olarak tanımlanır ve somut bileşenleri olur. Soyut ziyaretçi sınıfında hangi tür elemanların dolaşılacağını simgeleyen metotlar bulunur; ziyaretçi o tür elemanı ziyaret ettiğinde ne yapması gerektiği somut ziyaretçide kodlanır. Somut eleman alt sınıfı ise "beni şu metot üzerinden, şu işlemleri yapmak üzere ziyaret edebilirsin" diyerek ziyaretçinin ilgili metodunu çağırır.

Kalıp yapısı (ekrandan, 02:10:20):



7.7 Yeni eleman türü eklemenin zorluğu

Hoca bir zayıf yöne daha değindi: yeni bir somut eleman türü eklemek zordur. Şemaya bir `ConcreteElementC` eklenmesi gerektiğinde yalnız yeni bir kardeş sınıf eklemekle yetinilemez; hem soyut `Visitor` sınıfına hem de bu yüzden bütün somut ziyaretçi alt sınıflarına `VisitConcreteElementC` metodu eklenmelidir. Bir somut ziyaretçinin o yeni elemanla işi olmasa bile metot eklenip boş bırakılır; işi varsa zaten kodlanması gerekir, ama bu kez kodun yazılacağı yerler dağılır.

> **Hocanın vurgusu:** Yeni eleman türleri eklenmeyecek ama yeni işler ve sorumluluklar eklenecek bir parça-bütün yapısında tercih edilecek kalıp Visitor olabilir. İşler az değişecek ama eleman türleri sık sık farklılaşacaksa bu kalıbı kullanmak pek doğru değildir.

7.8 Etkileşim

Hoca kalıbın etkileşim şemasını (sequence diagram) çizerek anlattı. Nesne yapısında ziyaret süreci (şemada gösterilmeyen) bir metotla başlar. Nesne yapısı elemanlarından birine bir ziya-

retçi verip o elemanın bu ziyaretçiyi kabul etmesini (`Accept`) sağlar. Eleman "beni şu şekilde ziyaret et" diyerek ziyaretçinin ilgili `Visit` metodunu kendisini parametre vererek çağırır. Bu metot, o eleman üzerinde ne işlem yapılması gerekiyorsa onu yaptırır: örneğin `OperationA` çağrılır ve biter; başka işlemler (B, C gibi) varsa onlar da sırayla çağrılır ve döner. İşlemler bitince ziyaretçinin o elemanda yapacağı başka bir şey kalmaz ve `Visit` döner; ardından `Accept` de sona erer. Sonra benzer biçimde B elemanı dolaşılır: ona "ziyaretçiyi kabul et" denir; o da "ben B türündenim" diyerek ziyaretçinin `VisitConcreteElementB` metodunu kendisini vererek çağırır, B işlemi yapılır ve aynı sırayla dönülür.

> **Hocanın vurgusu:** Slayttaki etkileşim şemasında dönüş okları çizilmediği için akış çok net anlaşılmıyor. Her metot çağrısının bir dönüşü olacağı için her düz çizgili çağrı okunun karşılığında uygun bir yerde kesikli çizgili bir dönüş oku da gösterilmelidir; hoca üç çağrı için üç dönüş okunu tahtada ekleyerek gösterdi.

7.9 Ne zaman kullanılır?

Değişik arayüzlere sahip nesnelerden oluşan bir yapıda nesneler üzerinde farklı işlemler yapılması gerektiğinde bu kalıp kullanılır. Ama önemli bir şart daha vardır: nesne yapısındaki türler değişmeyecek ya da çok seyrek değişecektir; aksi halde fazladan çok kod eklemek gerekir.

8. Visitor Örneği: Proje Yönetim Yazılımı

8.1 Problem

Örnek "Applied Java Patterns" kitabından alındı; hoca yalnız görev tanımlarını değiştirdiğini, örneğin aynı olduğunu söyledi. Bir proje yönetim yazılımı hazırlanıyor. Modelin temel varlıkları şunlardır:

- Proje (`Project`): proje hiyerarşisinin kökü.
- Görev (`Task`): projede tanımlanan görevler. Görevler kendi içinde alt görevler ve ürünler içerebilir.
- Bağımlı görev (`DependentTask`): başlanmadan ya da bitmeden önce başka bir ya da birden fazla görevin yapılmasını gerektiren görev.
- Ürün (`Deliverable`): bir görev gerekiyorsa sonunda bir ya da birden fazla ürün üretebilir ya da hiç üretmeyebilir. Ürün, projenin ara çıktısı ya da sonucu olan herhangi bir kod ya da yapıttır (artifact): belge, ürün ya da başka bir şey olabilir.

8.2 Birinci çözüm

Proje kendi içinde görevler içerecek; projenin aşamalarında üretilmesi gereken ürünler olabilir. Bunların hepsi bir parça-bütün yapısında olduğu için ilk akla Composite gelir: proje kendi içinde proje öğeleri (`ProjectItem`) içerir ve projenin kendisi de `ProjectItem`'ın bir alt sınıfıdır.

Sınıf şeması (ekrandan, 02:20:00):

```
<<Abstract>> ProjectItem
- name, description : String
+ ProjectItem(String, String)
+ getName() : String            + setName(String)
+ getDescription() : String     + setDescription(String)
        ^
        +------------------------+---------------------------+
        |                        |                           |
Deliverable                Task                          Project  (ProjectItem'ları içerir)
- materialsCost,           - projectItems :              - projectItems :
  productionCost : double    ArrayList<ProjectItem>        ArrayList<ProjectItem>
+ Deliverable(String,       - timeRequired : double       + Project(String, String)
  String, Contact,          + Task(String, String,        + getProjectItems() : ArrayList<Proje
  double, double)             Contact, double)            + addProjectItem(ProjectItem)
+ getOwner() : Contact      + getProjectItems() : ...     + removeProjectItem(ProjectItem)
+ getMaterialsCost()        + addProjectItem(ProjectItem)
+ getProductionCost()       + removeProjectItem(ProjectItem)
```

```
+ setOwner(Contact)        + getOwner() : Contact
+ setMaterialsCost(double) + getTimeRequired() : double
+ setProductionCost(double)+ setOwner(Contact)
                           + setTimeRequired(double)

Contact: Deliverable ve Task'ın sahibi (owner, 1)

ProjectCalculator (Project'i tanır)          Report
+ calculateTimeRequired() : double           + Report()
- calculateTimeRequired(subTask : Task) : double   + main()
+ calculateCost() : double
- calculateCost(subTask : Task) : double
```

Her `ProjectItem`'ın adı ve açıklaması vardır; kurucuda bu üyeler ilklendirilir ve getter/setter metotları bulunur. Hoca bunun tam bir Composite olmadığını belirtti: Composite olsaydı `ProjectItem`'dan kendisine doğru "benim çocuklarım" anlamında bir-çok (elmaslı) ilişki olması gerekirdi. Visitor bir Composite üzerine de kurulabilir, normal bir parça-bütün ilişkisi üzerine de; Visitor illa Composite ile çalışacağım demez.

`Project`, proje öğelerini bir `ArrayList` içinde tutar; kurucuda ad ve açıklama verilir ve üst sınıfın kurucusu çağrılır. Öğeleri liste olarak döndüren, öğe ekleyen ve silen metotları vardır. `Task` da alt görevler ve ürünler içerebilir; bu yüzden onda da `ProjectItem` türünden elemanlar tutan bir `ArrayList` ve `double` türünde gereken süre (`timeRequired`) vardır. Görevin bir sorumlu kişisi (`Contact`) de vardır; hoca örneği daha fazla karmaşıklaştırmamak için bu sınıfı şemada büyütmediğini, yok gibi de düşünülebileceğini söyledi. `Deliverable`'da malzeme bedeli (`materialsCost`) ve üretim bedeli (`productionCost`) olmak üzere iki ayrı alan vardır: ürün için hem bir malzeme, ham madde gerekir hem de üretecek kişinin emeği, maaşı ya da kullanılan makinelerin harcadığı elektrik gibi bedeller söz konusudur. Kurucu bu iki bilgiyi ve üst sınıfın ad ile açıklamasını alır.

Bu yapıdan rapor hazırlanmak istenir: proje ne kadar zaman gerektiriyor ve kaça mal olacak? Bunun için `ProjectCalculator`'da `calculateTimeRequired` ve `calculateCost` metotları vardır. Alt yapıda, her öğenin alt öğelerinin süre ve bedellerini de hesaba katan `private` metotlar da bulunur; kullanıcı bunları doğrudan çağırmaz, ana metotları çağırır. `Report` sınıfı bir `ProjectCalculator` üzerinden hesaplatıp ekrana yazar.

8.3 İkinci çözüm: Visitor

Bir araya gelmiş nesnelerden oluşan bir koleksiyon ve onlar üzerinde yapılacak çeşitli raporlama işleri vardır: kâh zaman raporu, kâh maliyet raporu, kâh ileride istenebilecek başka rapor türleri. Ama yapının bileşenleri değişmeyecektir: proje her zaman alt proje öğeleri, görevler ve ürünlerden oluşacaktır. Farklı öğe türleri çıkmayacağı için Visitor kullanılabilir. Bu yüzden hem soyut `ProjectItem` sınıfına hem de bütün alt sınıflarına `accept` metodu konur.

Sınıf şeması (ekrandan, 02:25:20; önceki çözümle aynı olan üyeler gösterilmemiştir):

```
<<Abstract>> ProjectItem
- name, description : String
+ accept(ProjectVisitor)
        ^
        +----------------------+------------------------+
Deliverable               Task                        Project
+ accept(ProjectVisitor)  + accept(ProjectVisitor)    + accept(ProjectVisitor)
        :                         :                          :
        + - - - - - - - - - - > <<interface>> ProjectVisitor < - - +
                                  + visitDependentTask(DependentTask)
                                  + visitDeliverable(Deliverable)
                                  + visitTask(Task)
                                  + visitProject(Project)
                                          ^
                        +- - - - - - - - + - - - - - - - -+
                ProjectCostVisitor  <- - Report - ->  ProjectTimeVisitor
```

```
                                    + Report()
                                    + main()
```

`ProjectVisitor` arayüzünde her düğüm türü için bir `visit` metodu yazılmalıdır. Maliyet hesabı gerekiyorsa bir `ProjectCostVisitor`, zaman hesabı gerekiyorsa bir `ProjectTimeVisitor` nesnesi oluşturulup bütün yapıyı dolaşması sağlanır. Örneğin bir `Task`'ın `accept` metodu, bu iki ziyaretçiden ya da ileride eklenebilecek kardeş ziyaretçilerden herhangi birini `ProjectVisitor` türünden parametre olarak alır. Görev, `Task` sınıfında olduğu için `accept` içinde `p.visitTask(this)` der, yani kendi işaretçisini verir.

8.4 Kodların çalıştırılması

`Report` sınıfının kurucusunda örnek proje olarak bir yüksek lisans eğitimi tanımlanmıştır. Görevler dönemlerdir: iki dönem ders alınır, sonra yüksek lisans tezi dersi alınır. Her dönemde dört ders seçilir ve dersler ürün (deliverable) olarak tanımlanır; dönemler görevdir. Tez görevinin ürünleri tez metni ve tez kapsamında hazırlanan ulusal ya da uluslararası bir bildiri ya da makaledir. Hepsi proje öğesi olarak eklenir. Sahip (`Contact`) olarak `null` verilmiştir; hocaya göre kim olursa olsun bu işlerin yapılması gerekecektir.

1. çözümün `Report.java` dosyasından bir parça (ekrandan, 02:30:00):

```
    urun = new Deliverable("BLM 5139", "Çok Değişkenli Veri Analizi", null, 45, ...
    gorev.addProjectItem(urun);
    urun = new Deliverable("BLM 5188", "Seminer", null, 45, 30);
    gorev.addProjectItem(urun);
    project.addProjectItem(gorev);
    gorev = new Task("BLM 5198", "... Yüksek lisans tezi", null, 30);
    urun = new Deliverable("Tez metni", "Bez ciltli tez", null, 90, 270);
    gorev.addProjectItem(urun);
    urun = new Deliverable("Bildiri", "Ulusal veya uluslararası bildiri veya makale", null, 4.
    gorev.addProjectItem(urun);
    project.addProjectItem(gorev);
}

public static void main(String[] args) {
    Report rep = new Report();
    ProjectCalculator calc = new ProjectCalculator(rep.project);
    System.out.println("Proje adı ve açıklaması: " + rep.project.getName() + ": " + rep.proje.
    System.out.println("Proje süresi: " + calc.calculateTimeRequired() + " hafta.");
    System.out.println("Proje maliyeti: " + calc.calculateCost() + " saat çalışma.");
}
```

Hoca bir ders ürününün malzeme bedelinin 15 hafta boyunca haftada 3 saatten 45 ders saati olduğunu söyledi; maliyet, bu iki bedelin toplamı olarak hesaplanır.

`ProjectCalculator`'ın zaman hesabında kümülatif bir toplam tutulur ve her alt öğeye bakılır. Süre yalnız görevlerde vardır (ürünlerin süresi değil, malzeme ve üretim bedeli vardır), bu yüzden öğenin bir görev olup olmadığı denetlenir ve görevse süresi toplama eklenir. Public metot her öğe için `private` alt metodu çağırır; çünkü alt öğelerin de kendi alt öğeleri olabilir ve onların süreleri de hesaba katılmalıdır. Maliyet hesabı da aynı mantıkla, bu kez yalnız maliyet bileşenleri alınarak kümülatif toplama eklenerek yapılır. Program çalıştırıldığında projenin 60 hafta süreceği ve toplam 1174 saat çalışmaya mal olacağı yazıldı.

2. çözümde `Report`'un kurucusu aynıdır; fark `main` metodundaki çağırılış biçimindedir. Rapor oluşturulduktan sonra bir maliyet ziyaretçisi oluşturulur ve proje maliyeti için onun `getCost` metodu çağrılır; zaman için de bir zaman ziyaretçisi oluşturulup ona sorulur. 1. çözümde tek bir hesaplayıcı nesnesine iki soru ayrı ayrı soruluyordu; burada her ayrı ziyaretçi nesnesine kendisiyle ilgili hesabın sonucu sorulur. Visitor'da bunlar birleştirilmez, çünkü işin esprisi farklı ziyaretçilerin aynı yapıyı farklı amaçlarla dolaşabilmesidir. 2. çözüm çalıştırıldığında da proje adı ve açıklaması yazıldı, yine 60 hafta ve 1174 saat çalışma hesaplandı. Hoca, bir şey yanlış kodlansaydı ya yanlış sonuç çıkacağını ya da sonsuz döngüye girileceğini, iki çözümün aynı sonucu vermesinin altyapının düzgün kurulduğunu gösterdiğini söyledi.

8.5 İki çözümün karşılaştırılması

Başka bir şey hesaplanması gerekirse Visitor çözümünde yapılacak tek şey yeni bir ziyaretçi sınıfı eklemektir; başka hiçbir yere bir şey eklenmez. Hocaya göre bu, öbür çözüme göre daha derli toplu ve esnek bir çözümdür.

Hoca yalnız `main` metotlarını karşılaştırmanın ayırt edici olmayacağını, eski çözümdeki `ProjectCalculator` ile yeni çözümdeki `ProjectCostVisitor` ve `ProjectTimeVisitor`'ı karşılaştırmak gerektiğini söyledi. `ProjectCalculator` yaklaşık 50 satır, iki ziyaretçinin her biri yaklaşık 35 satırdır. Bir sorun çıktığında, eski çözümde 50 satır incelenir; yeni çözümde sorunun zamanda mı maliyette mi olduğu bilinip yalnız ilgili ziyaretçi, yani en fazla 35 satır incelenir (iki ziyaretçinin toplamı değil). Hoca bunun çok büyük bir fark yaratmayacağını, çünkü eski sürümde de zaman ve maliyet metotlarını ayrı ayrı ayırdığını kabul etti. Ama asıl fark şudur: yeni bir işlem gerektiğinde bu işlem bileşenlerin her birinin üstüne tek tek eklenmeyecektir. Hocaya göre buradaki "kötü" çözüm, kalıpları anlatırken verdiği kötü çözümlerden bir gömlek üstündür.

Eclipse'in iki yönlü karşılaştırmasıyla iki sınıf yan yana açıldı (ekrandan, 02:40:00):

```
package dp.visitor.solution1;

public class ProjectCalculator {
    private Project project;
    public ProjectCalculator(Project project) {
        this.project = project;
    }
    public double calculateTimeRequired( ) {
        double time = 0.0;
        for( ProjectItem item : project.getProje... ) {
            if( item instanceof Task ) {
                time += calculateTimeRequired((T...
            }
        }
        ...
```

```
package dp.visitor.solution2;

public class ProjectCostVisitor implements Pro... {
    private double cost = 0.0;

    public ProjectCostVisitor(Project project) {
        project.accept(this);
    }

    public void visitDependentTask(DependentTa... p) {
        visitTask(p);
    }
    ...
```

Eski çözüm daha karmaşıktır. Karmaşıklığın önemli kaynaklarından biri şudur: zaman hesaplanırken her düğümün değil yalnız görevlerin süresi vardır, bu yüzden önce düğümün görev olup olmadığına bakılır, öyleyse hesaba eklenir. Visitor uygulanan sürümde bu denetim yoktur; `ProjectTimeVisitor` ile karşılaştırıldığında da benzer içerik görülür ve orada da `instanceof` yoktur.

> **Hocanın vurgusu:** Kod yazarken kendinizi `instanceof` operatörünü kullanırken görüyorsanız, büyük ihtimalle kalıtım hiyerarşisini çok doğru kuramamışsınızdır ya da daha iyi kurabileceğinizin bir işaretidir. Bu, Visitor çözümünün vurgulanabilecek somut bir avantajıdır.

Hoca son olarak slayttaki bir ayrıntıyı düzeltti: `ProjectItem`'daki `accept` aslında soyut olmalıdır. Slaytlara yalnız UML kısımlarını koyduğunu, her şeyi PowerPoint'e yazmak zor olduğu için kodların kaynak dosyalardan açılıp incelenmesinin ve karşılaştırılmasının daha kolay olduğunu söyledi.

Örnek Sorular

1. (ekrandan, 00:37:10 ve 00:40:00) Uluslararası pazara hitap eden bir kişisel finans programı yazılıyor. Program değişik ülkelerdeki değişik vergi kurallarıyla işlem yapmalıdır; ana program `TaskController`'dır. Basit bir switch-case çözümünün sakıncası nedir ve hangi kalıpla nasıl bir tasarım yapılır?

> **Çözüm (derste verildi):** Switch-case çözümünde (`enum Country { TR, US, EU }`, `taxAmount` içinde `switch (code)`) gerçek uygulamada her case yüzlerce satır olur; yeni bir ülke için hem enum'a değer hem switch'e case eklenir ve metot giderek uzar, bakım pahalılaşır. Belli bir işin farklı yöntemlerle yapılması söz konusu olduğu için Strategy kullanılır: `<<interface>> TaxRule` (`taxAmount(income: double): double`), gerçeklemeleri `TRrules`, `EUrules`, `USrules`; `TaskController`'da `tax: TaxRule` ve `setTax(tax: TaxRule)`. Yeni ülke yalnız yeni bir sınıf demektir. Strateji sınıfları ayrıca Singleton yapılabilir.

2. (derste sözlü, 00:25:04) Strategy kalıbında herhangi bir anda bir stratejiden diğerine geçişi kim yönetir?

> **Çözüm (derste verildi):** Bundan sorumlu ayrı bir varlık olabilir: ayrı bir fabrika metodu ya da bir yapılandırma dosyasını okuyup stratejiyi Context'e setter ile veren bir kısım. Hangi somut stratejinin kullanılacağına Context de istekçi de karar verebilir. Soyut strateji sınıfına alt sınıfları bilen bir karar metodu koymak ise üst sınıfı alt sınıflara bağımlı kılacağı için önerilmez.

3. (derste sözlü, 00:32:27) Builder ile Strategy kalıplarının sınıf şemaları arasında fark var mıdır? İki kalıbı ne ayırır?

> **Çözüm (derste verildi):** Sınıf yapısı açısından fark yoktur (Director/RTF okuyucu ile Context, soyut builder/`TextConverter` ile soyut strateji, somut dönüştürücüler ile somut stratejiler karşılık gelir). Fark ayrıntıdadır: Builder'da Director bütün alt işleri bilir ve her alt işi adım adım ilgili alt sınıfa yaptırır; Strategy'de ise iş komple stratejiye verilir.

4. (derste sözlü, 00:41:33) Finans programında vergi hesabı dışında vergi indirimine başvurma, aile desteği, kredi borcu hesabı gibi ülkeden ülkeye değişen yeni işler eklenirse tasarım nasıl genişletilir?

> **Çözüm (derste verildi):** İki seçenek vardır: ya her iş için Strategy kalıbı ayrıca kurulur (ayrı üye alan, ayrı arayüz, ayrı somut gerçeklemeler), ya da yeni işler ilk strateji arayüzüne yeni metotlar olarak eklenir ve bir ülkenin bütün kuralları tek sınıfta toplanır. İkisi de geçerlidir; seçim tasarımcının tercihidir.

5. (ekrandan, 01:10:00) State ve Strategy kalıplarının farkları nelerdir?

> **Çözüm (derste verildi):** State'te Context'teki durum değiştikçe Context kendi `ConcreteState`'ini seçer; durum geçişleri açıkça (Context'e verilen komutlarla ya da `ConcreteState`'lerin içinden) yapılır ve sıktır; durum nesneleri Context'i daha sık sorgular. Strategy'de strateji geçişleri seyrektir; strateji nesneleri Context hakkında daha az şey bilir ve kendilerine gelen parametrelere göre işlem yapar. En önemli fark geçiş sıklığıdır.

6. (ekrandan, 01:20:00) Bir kişisel bilgi yöneticisi programında kayıtlar temiz (güncel) ya da kirli (değiştirilmiş ama kaydedilmemiş) olabilir. Kirliyken pencere kapatılırsa kullanıcıya kaydetmek isteyip istemediği sorulmalıdır. Bu davranış nasıl tasarlanır?

> **Çözüm (derste verildi):** "Durum" kelimesi yerine oturduğu için State kalıbı kullanılır. `PIMApplication`'da `+CLEAN: State`, `+DIRTY: State` (public final static), `-currentState: State`, `getCurrentState()`, `setCurrentState(State)` bulunur. Soyut `State` ortak `save(File, Serializable)` ve varsayılan `edit()` içerir. `CleanState.edit()` uygulamayı kirli duruma geçirip `super.edit()` çağırır; kaydetme sonrasında uygulama temiz duruma geçer. `DirtyState` ek bir şey içermeyen bir marker sınıftır. Uygulama düzenleme ve kaydetme işlerini o anki duruma yaptırır.

7. (ekrandan, 01:36:10) Uygulamamız veritabanından kayıt çekecek ve farklı VTYS'lerle çalışabilmelidir. Bağlantı ve SELECT komutlarının biçimi VTYS'den VTYS'ye değişir, ama iş her

VTYS'de aynı beş adımda yürür. Hangi kalıp kullanılır, neden Strategy değil?

> **Çözüm (derste verildi):** Template Method kullanılır, çünkü farklı işlerin yanında yürütülecek ortak işlemler de vardır; Strategy'de ise işin hepsini strateji yapar. Soyut `QueryTemplate`'te `+doQuery(DBname, querySpec: String) {final}` şablon metottur; `~formatConnect(DBname: String) {abstract}` ve `~formatSelect(querySpec: String) {abstract}` alt sınıflara (`OracleQT`, `SQLServerQT`, `GraphDBQT`) bırakılır. `QueryControl` kurucuda bir `QueryTemplate` alır (`-qt: QueryTemplate`). Biçimlendirme metotları iç çalışmaya ilişkin olduğu için dışarıya açılmaz; paket görünürlüğü (ya da protected) kullanılır.

8. (derste sözlü, 02:11:13) Visitor kalıbı kurulmuş bir yapıya yeni bir somut eleman türü (`ConcreteElementC`) eklenmesi gerekirse ne olur? Bundan kalıbın kullanım alanı için ne sonuç çıkar?

> **Çözüm (derste verildi):** Yalnız yeni bir kardeş eleman sınıfı eklemek yetmez; soyut `Visitor`'a ve bu yüzden bütün somut ziyaretçilere `VisitConcreteElementC` metodu eklenmelidir. Bir ziyaretçinin bu elemanla işi yoksa bile metot boş olarak eklenir; işi varsa kod dağınık yerlere yazılır. Sonuç: Visitor, eleman türleri değişmeyecek ama yeni işler eklenecek yapılarda uygundur; eleman türleri sık değişecekse uygun değildir.

9. (ekrandan, 02:20:00 ve 02:25:20) Bir proje yönetim yazılımında proje, görev, bağımlı görev ve ürün öğeleri vardır. Bu yapıdan önce zaman ve maliyet raporları, ileride başka raporlar da üretilecektir; öğe türleri ise değişmeyecektir. Esnek bir tasarım nasıl kurulur?

> **Çözüm (derste verildi):** Visitor kullanılır. Soyut `ProjectItem` ve alt sınıfları `Deliverable`, `Task`, `Project` birer `accept(ProjectVisitor)` metodu alır; örneğin `Task` içinde `p.visitTask(this)` çağrılır. `<<interface>> ProjectVisitor`'da `visitDependentTask(DependentTask)`, `visitDeliverable(Deliverable)`, `visitTask(Task)`, `visitProject(Project)` metotları vardır; `ProjectCostVisitor` ve `ProjectTimeVisitor` bunu gerçekler ve `Report` ikisini kullanır. Yeni bir rapor yalnız yeni bir ziyaretçi sınıfı demektir ve 1. çözümdeki `instanceof Task` denetimine gerek kalmaz.

Dersin Toparlaması

Bu derste davranışsal kalıplardan dördü işlendi. Strategy, aynı işin farklı algoritmalarını istekçiden soyutlayarak algoritmanın çalışma anında değiştirilmesini sağlar; vergi örneğinde switch-case yerine bir `TaxRule` arayüzü ve ülke başına bir sınıf kuruldu, strateji sınıflarının iyi birer Singleton adayı olduğu gösterildi. Hoca Strategy'yi sınıf şeması neredeyse aynı olan Builder ile karşılaştırarak kalıpları ayıranın şema değil, işin nasıl yürütüldüğüne ilişkin ayrıntılar olduğunu vurguladı. State, nesnenin davranışını iç durumuna göre bütüncül olarak değiştirir; Strategy'den en önemli farkı durum geçişlerinin sık, strateji geçişlerinin seyrek olmasıdır. Sonlu durum makinelerine benzeyen bu yapı, PIM programının temiz ve kirli durumlarıyla örneklendi.

Template Method, alternatif algoritmaların ortak adımlarını üst sınıftaki `final` bir şablon metotta toplayıp değişen adımları soyut metotlarla alt sınıflara bırakır; veritabanından bağımsız sorgu örneğinde "ortak işlemler de var" ifadesinin Strategy yerine Template Method'u işaret ettiği görüldü. Son olarak Visitor, bir nesne yapısının elemanları üzerinde yapılacak işlemleri, elemanların sınıflarını değiştirmeden ayrı ziyaretçi sınıflarında toplar; yeni işlem eklemek kolay, yeni eleman türü eklemek zordur. Proje yönetim örneğinde Visitor çözümü `instanceof` denetimini ortadan kaldırdı. Davranışsal kalıplardan Mediator bu derste işlenmedi ve sonraya kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.